Centro Médico

Sistema de Gestión — Documentación Técnica

Entrega 3

Validación de datos · API REST · Seguridad

Taller de Programación

Octubre de 2026

Índice

1. Qué es el sistema y qué resuelve
2. Arquitectura general
3. Tecnologías utilizadas
4. Estructura de carpetas
5. El recorrido completo de una petición
6. Capa de modelo: las entidades
7. Capa de repositorios: el acceso a datos
8. Los DTO y la validación de datos
9. Capa de servicios: las reglas de negocio
10. Capa de controladores: la API REST
11. El manejo centralizado de errores
12. Seguridad: contraseñas y tokens
13. El frontend y cómo consume la API
14. Tabla completa de endpoints
15. La base de datos
16. Preguntas probables en la defensa

1. Qué es el sistema y qué resuelve

El sistema administra la operación diaria de un centro médico. Permite registrar pacientes, dar de alta profesionales y sus especialidades, agendar turnos y registrar la historia clínica de cada paciente.

El proyecto está formado por dos partes que cumplen roles distintos:

- **El frontend:** las páginas que ve el usuario, hechas con HTML, CSS, Bootstrap y JavaScript.
- **El backend:** un programa escrito en Java con Spring Boot que guarda la información en una base de datos PostgreSQL y la entrega cuando el frontend se la pide.

> **El punto central de esta entrega.** Antes, las páginas mostraban datos escritos a mano dentro del propio archivo HTML: si se recargaba la página, todo seguía igual, porque nada se guardaba. Ahora los datos están en una base de datos real y las páginas los piden al servidor. Esa comunicación es lo que se implementó en esta etapa.

Qué se puede hacer

Módulo	Funciones
Acceso	Inicio de sesión con usuario y contraseña, con tres roles distintos
Pacientes	Alta, modificación, baja y búsqueda por DNI, nombre o apellido
Especialidades	Alta, modificación, baja y activación/desactivación
Profesionales	Alta, modificación, baja y filtrado por especialidad
Turnos	Registro, modificación, cancelación y cambio de estado
Agenda	Vista semanal de los turnos de cada profesional
Historia clínica	Registro de atenciones e historial cronológico por paciente

2. Arquitectura general

El backend está organizado en **capas**. Cada capa tiene una única responsabilidad y solo se comunica con la siguiente. Este esquema se conoce como **Controlador → Servicio → Repositorio**.

```
NAVEGADOR (HTML + JavaScript)
      |
      |  petición HTTP con datos en formato JSON
      v
+-----------------------------+
|  CONTROLADOR  (@RestController)
|  Recibe la petición y devuelve la respuesta
+-----------------------------+
      |
      v
+-----------------------------+
|  SERVICIO  (@Service)
|  Aplica las reglas del negocio
+-----------------------------+
      |
      v
+-----------------------------+
|  REPOSITORIO  (JpaRepository)
|  Consulta y guarda en la base
+-----------------------------+
      |
      v
BASE DE DATOS (PostgreSQL)
```

Por qué separar en capas

La separación permite que cada parte cambie sin afectar a las demás:

- Si se cambia de PostgreSQL a otra base de datos, solo se modifica la capa de repositorios.
- Si cambia una regla del negocio (por ejemplo, permitir turnos los sábados), solo se modifica el servicio. El controlador ni se entera.
- Si se quisiera agregar una aplicación de celular, usaría los mismos controladores y servicios: solo cambiaría la interfaz.

Además, esta separación permite **probar la lógica de negocio sin base de datos**, porque el servicio no sabe de dónde vienen los datos.

3. Tecnologías utilizadas

Capa	Tecnología	Para qué se usa
Lenguaje	Java 25	Todo el backend
Framework	Spring Boot 4.1.2	Levanta el servidor y conecta los componentes
Acceso a datos	Spring Data JPA (Hibernate)	Traduce objetos Java a tablas SQL
Base de datos	PostgreSQL 17	Almacena la información
Seguridad	Spring Security + JWT	Login, tokens y permisos por rol
Validación	Jakarta Bean Validation	Verifica el formato de los datos
Frontend	HTML5, CSS3, Bootstrap 5.3	Las páginas y su diseño
Comunicación	JavaScript con `fetch`	Pide los datos al backend
Dependencias	Maven	Descarga las librerías y compila

Qué es Spring Boot

Spring Boot es un framework que se encarga del trabajo repetitivo de armar una aplicación web: levanta un servidor, conecta la base de datos y arma los objetos. El programador escribe las clases con anotaciones (`@RestController`, `@Service`) y Spring las detecta y las conecta automáticamente. A eso se lo llama **inyección de dependencias**.

Qué es JPA e Hibernate

JPA permite trabajar con la base de datos usando objetos Java en vez de escribir SQL a mano. Una clase Java representa una tabla y cada objeto una fila. Hibernate es la implementación que traduce esas operaciones a SQL real.

> **Ventaja concreta:** al no escribir SQL manualmente, el sistema queda automáticamente protegido contra **inyección SQL**, porque todos los valores viajan como parámetros y nunca como texto pegado a la consulta.

4. Estructura de carpetas

```
centro-medico/
├── pom.xml                          Dependencias y configuración de Maven
└── src/main/
    ├── java/com/centromedico/
    │   ├── CentroMedicoApplication.java    Clase que arranca todo
    │   ├── config/
    │   │   ├── SecurityConfig.java         Reglas de seguridad
    │   │   └── DataSeeder.java             Datos de ejemplo iniciales
    │   ├── controller/                     Los endpoints de la API
    │   ├── dto/                            Datos de entrada/salida + validaciones
    │   ├── exception/                      Errores propios y manejador global
    │   ├── model/                          Entidades = tablas de la base
    │   ├── repository/                     Consultas a la base
    │   ├── security/                       Generación y control de los tokens
    │   └── service/                        Reglas de negocio
    └── resources/
        ├── application.properties          Configuración (base de datos, puerto)
        └── static/                         EL FRONTEND
            ├── *.html                      Las páginas
            ├── css/                        Hojas de estilo
            ├── img/                        Imágenes
            └── js/                         JavaScript que consume la API
```

> **Por qué el frontend está dentro de `resources/static`:** Spring Boot publica automáticamente todo lo que esté en esa carpeta. Así el mismo servidor entrega las páginas y atiende la API, y se levanta todo con un solo comando. Además evita el problema de **CORS**, que aparece cuando el frontend y el backend están en direcciones distintas.

Cómo se ejecuta

```
cd centro-medico
mvn spring-boot:run
```

Luego se abre el navegador en `http://localhost:8080`.

> **Importante:** las páginas ya no funcionan abriendo el archivo HTML con doble clic. Deben abrirse por `http://localhost:8080`, porque necesitan al servidor para obtener los datos.

5. El recorrido completo de una petición

Esta sección sigue el camino completo de una operación real: **guardar un paciente nuevo**. Es el mejor resumen de cómo funciona todo el sistema.

1. **El usuario completa el formulario** en `paciente-form.html` y presiona Guardar.

2. **El JavaScript arma el pedido.** El archivo `js/paciente-form.js` lee los campos y los convierte en un objeto:

```
const datos = {
    dni: formulario.dni.value.trim(),
    nombre: formulario.nombre.value.trim(),
    apellido: formulario.apellido.value.trim(),
    fechaNacimiento: formulario.fechaNacimiento.value || null,
    ...
};

await api.post("/pacientes", datos);
```

3. **Se envía al servidor.** La función `api.post` usa `fetch`, convierte el objeto a texto JSON y le agrega el token de sesión:

```
const respuesta = await fetch("/api/pacientes", {
    method: "POST",
    headers: {
        "Content-Type": "application/json",
        "Authorization": "Bearer " + token
    },
    body: JSON.stringify(datos)
});
```

4. **El filtro de seguridad revisa el token.** Antes de llegar al controlador, `JwtAuthenticationFilter` verifica que el token sea válido. Si falta o está vencido, responde **401** y la petición no avanza.

5. **El controlador recibe los datos.** Spring convierte el JSON en un objeto Java automáticamente. La anotación `@Valid` dispara las validaciones de formato:

```
@PostMapping
public ResponseEntity<PacienteResponse> crear(@Valid @RequestBody PacienteRequest peticion)
{
    return ResponseEntity.status(HttpStatus.CREATED).body(pacienteService.crear(peticion));
}
```

Si algún campo no cumple su regla, se corta acá y devuelve **400** con el detalle de cada campo.

6 **El servicio aplica las reglas del negocio.** Verifica que no exista otro paciente con ese DNI:

```
if (pacienteRepository.existsByDni(peticion.dni())) {
    throw new ReglaNegocioException("Ya existe un paciente registrado con el DNI " +
peticion.dni());
}
```

7 **El repositorio guarda en la base.** `pacienteRepository.save(paciente)` genera el `INSERT` de SQL.

8 **Vuelve la respuesta.** El servicio convierte la entidad en un `PacienteResponse` y el controlador lo devuelve como JSON con el código **201 Created**.

9 **El navegador muestra el resultado.** El JavaScript muestra el aviso verde de confirmación y redirige al listado.

Resumen de la respuesta del paso a paso: el dato pasó por cuatro controles antes de guardarse — token válido, formato correcto, regla de negocio cumplida y restricción de la base de datos.

6. Capa de modelo: las entidades

Las clases de `model/` representan las tablas. Hibernate las lee al arrancar y **crea las tablas automáticamente**, por eso no hace falta escribir SQL para armar la base.

Ejemplo: la entidad Paciente

```
@Entity
@Table(name = "pacientes")
public class Paciente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 15)
    private String dni;

    @Column(nullable = false, length = 80)
    private String nombre;

    @Column(name = "fecha_nacimiento", nullable = false)
    private LocalDate fechaNacimiento;

    @Column(nullable = false)
    private boolean activo = true;

    // getters y setters...
}
```

Qué significa cada anotación

Anotación	Qué hace
`@Entity`	Marca la clase como una tabla de la base
`@Table(name="...")`	Indica el nombre de la tabla
`@Id`	Señala la clave primaria
`@GeneratedValue`	La base genera el número solo, de forma automática
`@Column(unique=true)`	No permite valores repetidos
`@Column(nullable=false)`	El campo es obligatorio
`@ManyToOne`	Relación: muchos registros apuntan a uno

Las relaciones entre tablas

Un profesional pertenece a una especialidad. Eso se expresa así:

```
@ManyToOne(fetch = FetchType.EAGER, optional = false)
@JoinColumn(name = "especialidad_id", nullable = false)
private Especialidad especialidad;
```

En la base esto genera una columna `especialidad_id` con una **clave foránea**. En el código Java, en cambio, se accede al objeto completo: `profesional.getEspecialidad().getNombre()`.

Una restricción importante en los turnos

```
@Table(name = "turnos", uniqueConstraints = @UniqueConstraint(
        name = "uk_turno_profesional_fecha_hora",
        columnNames = {"profesional_id", "fecha", "hora"}))
```

> **Qué logra esto.** La propia base de datos impide que un profesional tenga dos turnos en la misma fecha y hora. Aunque alguien intentara saltearse la aplicación y escribir directo en la base, no podría. Es una **segunda línea de defensa** además de la validación del servicio.

Los enumerados

Para los valores fijos se usan `enum`, que evitan errores de tipeo:

```
public enum EstadoTurno { PENDIENTE, ATENDIDO, CANCELADO }

public enum Rol { ADMIN, RECEPCION, PROFESIONAL }
```

En la entidad se guardan como texto legible gracias a `@Enumerated(EnumType.STRING)`. Si se guardaran como número, al agregar un valor nuevo se podrían corromper los datos existentes.

7. Capa de repositorios: el acceso a datos

Los repositorios son **interfaces**: se declara qué consulta se necesita y Spring Data escribe la implementación automáticamente.

```
public interface PacienteRepository extends JpaRepository<Paciente, Long> {

    Optional<Paciente> findByDni(String dni);

    boolean existsByDni(String dni);

    boolean existsByDniAndIdNot(String dni, Long id);
}
```

Al extender `JpaRepository` ya se heredan gratis las operaciones básicas: `save()`, `findById()`, `findAll()`, `delete()` y `count()`.

Consultas derivadas del nombre del método

Spring Data interpreta el nombre del método y arma el SQL solo:

Método	Consulta que genera
`findByDni(dni)`	Busca el paciente con ese DNI
`existsByDni(dni)`	Devuelve verdadero o falso si existe
`existsByDniAndIdNot(dni, id)`	Si existe ese DNI en *otro* registro
`countByEspecialidadId(id)`	Cuenta los profesionales de esa especialidad

> **Para qué sirve `existsByDniAndIdNot`.** Al editar un paciente hay que verificar que el DNI no lo tenga *otro*. Si solo se usara `existsByDni`, al guardar sin cambiar el DNI el sistema encontraría al propio paciente y daría un error falso de duplicado.

Consultas escritas a mano

Para las búsquedas se usa `@Query` con JPQL, que es parecido a SQL pero trabaja con clases en vez de tablas:

```
@Query("""
        SELECT p FROM Paciente p
        WHERE p.activo = true
          AND (LOWER(p.nombre) LIKE LOWER(:patron)
               OR LOWER(p.apellido) LIKE LOWER(:patron)
               OR p.dni LIKE :patron)
        ORDER BY p.apellido ASC, p.nombre ASC
        """)
List<Paciente> buscar(@Param("patron") String patron);
```

El servicio arma el patrón de búsqueda antes de llamarla:

```
private String patron(String texto) {
    return (texto == null || texto.isBlank()) ? "%" : "%" + texto.trim() + "%";
}
```

Si no se escribió nada, el patrón es `%`, que en SQL significa "cualquier valor", y la consulta devuelve todos los registros. Así una sola consulta sirve tanto para listar como para buscar.

> **Un problema real que hubo que resolver.** La primera versión usaba `(:texto IS NULL OR ...)` para distinguir si había filtro. PostgreSQL no lograba deducir el tipo de ese parámetro, lo interpretaba como datos binarios y fallaba con el error `no existe la función lower(bytea)`. La solución fue armar el patrón en Java y eliminar la comparación contra nulo.

8. Los DTO y la validación de datos

DTO significa *Data Transfer Object*: un objeto que define exactamente qué datos entran y qué datos salen de la API. Están implementados como `record`, una forma corta de declarar clases que solo guardan datos.

Por qué no se usan las entidades directamente

- **Seguridad:** la entidad `Usuario` contiene la contraseña cifrada. Si se devolviera tal cual, ese dato saldría en la respuesta.
- **Campos calculados:** el DTO puede incluir la edad del paciente, que no está guardada en ninguna columna: se calcula a partir de la fecha de nacimiento.
- **Independencia:** se puede cambiar la estructura de las tablas sin romper lo que espera el frontend.

Las validaciones de formato

```
public record PacienteRequest(

        @NotBlank(message = "El DNI es obligatorio")
        @Pattern(regexp = "^[0-9]{7,9}$",
                 message = "El DNI debe tener entre 7 y 9 dígitos, sin puntos")
        String dni,

        @NotBlank(message = "El nombre es obligatorio")
        @Size(max = 80)
        @Pattern(regexp = "^[A-Za-zÁÉÍÓÚáéíóúÑñÜü ']+$",
                 message = "El nombre solo puede contener letras")
        String nombre,

        @NotNull(message = "La fecha de nacimiento es obligatoria")
        @Past(message = "La fecha de nacimiento debe ser anterior a hoy")
        LocalDate fechaNacimiento,

        @Email(message = "El email no tiene un formato válido")
        String email

) {}
```

Las anotaciones usadas

Anotación	Qué verifica
`@NotBlank`	Que no esté vacío ni sean solo espacios
`@NotNull`	Que el valor exista
`@Size(max=80)`	El largo máximo
`@Pattern(regexp=...)`	Que coincida con un patrón de caracteres
`@Email`	Formato de correo electrónico
`@Past`	Que la fecha sea anterior a hoy
`@PastOrPresent`	Que no sea una fecha futura

Las expresiones regulares explicadas

Patrón	Significado
`^[0-9]{7,9}$`	Solo dígitos, entre 7 y 9. El `^` es el inicio y `$` el final, así que no admite nada más
`^[A-Za-zÁÉÍÓÚáéíóúÑñÜü']+$`	Solo letras, incluidas las acentuadas y la ñ, más espacios y apóstrofo para apellidos como D'Angelo
`^$\|^(A\|B\|AB\|O)[+-]$`	Vacío, o un grupo sanguíneo válido. La barra `|` significa "o"

Las dos clases de validación

Esta distinción es importante para la defensa:

	Validación de formato	Regla de negocio
Dónde está	En el DTO, con anotaciones	En el servicio, con código Java
Qué responde	¿El dato tiene la forma correcta?	¿Tiene sentido en un centro médico?
Necesita la base	No	Sí
Código HTTP	400	409
Ejemplo	El DNI tiene 8 dígitos	Ese DNI ya está registrado

La razón de separarlas es que el formato se verifica mirando solo el dato, mientras que la regla de negocio necesita consultar qué hay guardado.

9. Capa de servicios: las reglas de negocio

Los servicios concentran las decisiones del sistema. Un controlador nunca decide nada: delega en el servicio.

Las reglas implementadas

Regla	Dónde está
No puede haber dos pacientes con el mismo DNI	PacienteService
No puede repetirse la matrícula de un profesional	ProfesionalService
Un profesional no puede tener dos turnos a la misma hora	TurnoService
No se pueden registrar turnos en fechas pasadas	TurnoService
No se asignan turnos a profesionales inactivos	TurnoService
Un turno cancelado no se puede modificar	TurnoService
Un turno atendido no vuelve a pendiente	TurnoService
No se borra una especialidad con profesionales	EspecialidadService
Los pacientes con historial se dan de baja lógica	PacienteService

Ejemplo: el registro de un turno

```
@Transactional
public TurnoResponse crear(TurnoRequest peticion) {

    Paciente paciente = buscarPacientePorDni(peticion.pacienteDni());
    Profesional profesional = profesionalService.buscarEntidad(peticion.profesionalId());

    validarProfesionalActivo(profesional);
    validarNoEsPasado(peticion.fecha(), peticion.hora());

    boolean ocupado = turnoRepository.existsByProfesionalIdAndFechaAndHoraAndEstadoNot(
            profesional.getId(), peticion.fecha(), peticion.hora(), EstadoTurno.CANCELADO);

    if (ocupado) {
        throw new ReglaNegocioException("El profesional ya tiene un turno asignado el "
                + peticion.fecha() + " a las " + peticion.hora().format(FORMATO_HORA));
    }

    Turno turno = new Turno();
    turno.setPaciente(paciente);
    turno.setProfesional(profesional);
    turno.setFecha(peticion.fecha());
    turno.setHora(peticion.hora());
    turno.setEstado(EstadoTurno.PENDIENTE);

    return aRespuesta(turnoRepository.save(turno));
}
```

> **Detalle del control de superposición.** La consulta termina en `EstadoNot(CANCELADO)` : los turnos cancelados se ignoran. Si alguien cancela un turno de las 9:00, ese horario queda libre para otro paciente. Sin esa condición, un turno cancelado bloquearía el horario para siempre.

Qué significa @Transactional

Agrupa todas las operaciones del método en una **transacción**: o se completan todas, o no se aplica ninguna. Si el método falla a la mitad, la base vuelve a su estado anterior y no quedan datos a medio guardar.

Cuando el método solo consulta se usa `@Transactional(readOnly = true)`, que le avisa a la base que no habrá modificaciones y le permite optimizar la lectura.

La baja lógica

```
@Transactional
public void eliminar(Long id) {
    Paciente paciente = buscarEntidad(id);

    boolean tieneHistorial = turnoRepository.countByPacienteId(id) > 0
            || atencionRepository.countByPacienteId(id) > 0;

    if (tieneHistorial) {
        paciente.setActivo(false);          // se oculta, no se borra
        pacienteRepository.save(paciente);
    } else {
        pacienteRepository.delete(paciente); // sin historial, se borra de verdad
    }
}
```

Borrar un paciente con atenciones registradas destruiría información médica. Por eso en ese caso se lo marca como inactivo: desaparece de los listados pero su historia clínica queda intacta.

10. Capa de controladores: la API REST

Los controladores son la puerta de entrada. Cada uno atiende un recurso.

```
@RestController
@RequestMapping("/api/pacientes")
public class PacienteController {

    private final PacienteService pacienteService;

    public PacienteController(PacienteService pacienteService) {
        this.pacienteService = pacienteService;
    }

    @GetMapping
    public List<PacienteResponse> listar(@RequestParam(required = false) String buscar) {
        return pacienteService.listar(buscar);
    }

    @GetMapping("/{id}")
    public PacienteResponse obtener(@PathVariable Long id) {
        return pacienteService.obtenerPorId(id);
    }

    @PostMapping
    public ResponseEntity<PacienteResponse> crear(@Valid @RequestBody PacienteRequest
peticion) {
        return
ResponseEntity.status(HttpStatus.CREATED).body(pacienteService.crear(peticion));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable Long id) {
        pacienteService.eliminar(id);
        return ResponseEntity.noContent().build();
    }
}
```

Las anotaciones del controlador

Anotación	Función
`@RestController`	Las respuestas se devuelven como JSON
`@RequestMapping`	La dirección base de todo el controlador
`@GetMapping`	Atiende peticiones de consulta
`@PostMapping`	Atiende la creación de registros
`@PutMapping`	Atiende la modificación completa
`@PatchMapping`	Modifica solo una parte (el estado del turno)
`@DeleteMapping`	Atiende la eliminación
`@PathVariable`	Toma un valor de la dirección: `/pacientes/5`
`@RequestParam`	Toma un valor de la consulta: `?buscar=perez`
`@RequestBody`	Convierte el JSON recibido en un objeto Java
`@Valid`	Ejecuta las validaciones del DTO

Qué es REST

REST es un estilo para diseñar APIs. Sus ideas principales son:

- Cada recurso tiene su propia dirección: `/api/pacientes`
- La acción la indica el verbo HTTP, no la dirección. No existe `/api/borrarPaciente`: se usa `DELETE` sobre `/api/pacientes/5`
- El servidor no guarda estado entre peticiones
- Se responde con los códigos HTTP correctos

Los códigos de respuesta usados

Código	Nombre	Cuándo se devuelve
200	OK	La consulta o modificación salió bien
201	Created	Se creó un registro nuevo
204	No Content	Se eliminó correctamente, no hay nada que devolver
400	Bad Request	Los datos enviados no cumplen el formato
401	Unauthorized	Falta el token, está vencido o las credenciales son incorrectas
403	Forbidden	El usuario está identificado pero su rol no tiene permiso
404	Not Found	El registro pedido no existe
409	Conflict	Se violó una regla de negocio

> **Diferencia entre 401 y 403.** El 401 significa "no sé quién sos": falta el token o no es válido. El 403 significa "sé quién sos, pero no podés hacer esto": por ejemplo, un usuario de recepción intentando borrar un paciente.

11. El manejo centralizado de errores

Sin un manejo central, un error devolvería una página HTML de error de Java, que el JavaScript no puede interpretar. La clase `GlobalExceptionHandler` resuelve esto: captura todos los errores y los convierte en JSON con un formato uniforme.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> manejarValidacion(MethodArgumentNotValidException ex)
{

        Map<String, String> errores = new LinkedHashMap<>();
        for (FieldError error : ex.getBindingResult().getFieldErrors()) {
            errores.putIfAbsent(error.getField(), error.getDefaultMessage());
        }

        return ResponseEntity.badRequest().body(ErrorResponse.deValidacion(
                HttpStatus.BAD_REQUEST.value(),
                "Datos inválidos",
                "Revise los campos marcados e intente nuevamente",
                errores));
    }

    @ExceptionHandler(RecursoNoEncontradoException.class)
    public ResponseEntity<ErrorResponse> manejarNoEncontrado(RecursoNoEncontradoException ex)
{
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(...);
    }

    @ExceptionHandler(ReglaNegocioException.class)
    public ResponseEntity<ErrorResponse> manejarReglaNegocio(ReglaNegocioException ex) {
        return ResponseEntity.status(HttpStatus.CONFLICT).body(...);
    }
}
```

`@RestControllerAdvice` hace que esta clase vigile a todos los controladores. Cada `@ExceptionHandler` atiende un tipo de error y le asigna su código HTTP.

El formato de la respuesta de error

```
{
  "timestamp": "2026-10-25T02:19:24",
  "status": 400,
  "error": "Datos inválidos",
  "mensaje": "Revise los campos marcados e intente nuevamente",
  "errores": {
    "dni": "El DNI debe tener entre 7 y 9 dígitos, sin puntos",
    "nombre": "El nombre solo puede contener letras",
    "fechaNacimiento": "La fecha de nacimiento debe ser anterior a hoy"
  }
}
```

El objeto `errores` tiene el nombre del campo y su mensaje. El frontend lo recorre y marca cada campo del formulario:

```
Object.entries(errores).forEach(([campo, mensaje]) => {
    const input = formulario.querySelector(`[name="${campo}"]`);
    input.classList.add("is-invalid");

    const detalle = document.createElement("div");
    detalle.className = "invalid-feedback";
    detalle.textContent = mensaje;
    input.insertAdjacentElement("afterend", detalle);
});
```

> **Por qué los nombres coinciden.** El atributo `name` de cada campo del HTML es igual al del DTO en Java. Esa coincidencia es la que permite que el mensaje aparezca exactamente debajo del campo que corresponde.

12. Seguridad: contraseñas y tokens

Las contraseñas nunca se guardan tal cual

Se almacena un **hash BCrypt**: un texto del cual no se puede volver atrás para obtener la contraseña original.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Al iniciar sesión el sistema **no compara contraseñas**: aplica el mismo procedimiento a lo que se escribió y compara los resultados.

```
if (!passwordEncoder.matches(peticion.password(), usuario.getPassword())) {
    throw new CredencialesInvalidasException("Usuario o contraseña incorrectos");
}
```

> **Qué protege esto.** Si alguien robara la base de datos, no obtendría ninguna contraseña. BCrypt además agrega un valor aleatorio llamado *salt*, así que dos usuarios con la misma contraseña tienen hashes distintos.

Qué es un JWT

JWT significa *JSON Web Token*. Es un texto firmado que identifica al usuario y viaja en cada petición. Tiene tres partes separadas por puntos:

```
eyJhbGciOiJIUzUxMiJ9 . eyJzdWIiOiJhZG1pbiIsInJvbCI6IkFETUlOIn0 . H7MqB5VlAEcjZtv0...
    [ cabecera ]              [ datos: usuario y rol ]            [ firma ]
```

Parte	Contenido
Cabecera	El algoritmo con el que se firmó
Datos	El usuario, su rol y la fecha de vencimiento
Firma	Un código generado con la clave secreta del servidor

> **El token está firmado, no encriptado.** Cualquiera puede leer lo que dice adentro, pero nadie puede modificarlo sin romper la firma, porque la clave secreta la tiene solo el servidor. Si se cambia una sola letra, el sistema lo rechaza. Por eso en el token nunca se guarda información sensible.

Cómo se genera

```
public String generarToken(String username, String rol) {
    Date ahora = new Date();
    return Jwts.builder()
            .subject(username)
            .claim("rol", rol)
            .issuedAt(ahora)
            .expiration(new Date(ahora.getTime() + expiracionMs))
            .signWith(clave)
            .compact();
}
```

Cómo se verifica en cada petición

La clase `JwtAuthenticationFilter` se ejecuta antes que cualquier controlador:

```
String encabezado = request.getHeader("Authorization");

if (encabezado != null && encabezado.startsWith("Bearer ")) {

    Claims datos = jwtService.validarYObtenerDatos(encabezado.substring(7));

    if (datos != null) {
        String username = datos.getSubject();
        String rol = datos.get("rol", String.class);

        var autoridades = List.of(new SimpleGrantedAuthority("ROLE_" + rol));
        var autenticacion = new UsernamePasswordAuthenticationToken(username, null,
autoridades);

        SecurityContextHolder.getContext().setAuthentication(autenticacion);
    }
}

filterChain.doFilter(request, response);
```

Si el token es válido, el usuario queda identificado para el resto del procesamiento. Si no lo es, la petición sigue sin autenticación y termina siendo rechazada con 401.

El flujo completo del login

1. El usuario envía sus credenciales a `/api/auth/login`.
2. El servidor busca el usuario y compara el hash de la contraseña.
3. Si coincide, genera un token firmado con el usuario y su rol.
4. El navegador lo guarda en `localStorage`.
5. En cada pedido posterior lo envía en la cabecera `Authorization`.

> 6 El filtro lo valida antes de permitir el acceso.

> 7 A las 8 horas vence y hay que volver a iniciar sesión.

Sesiones sin estado

```
.sessionManagement(sesion -> sesion.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
```

El servidor **no guarda ninguna sesión en memoria**. Cada petición se valida sola, por su firma. Esto se llama *stateless* y tiene una ventaja concreta: si hubiera varios servidores, cualquiera podría atender cualquier petición, porque ninguno necesita recordar al usuario.

Los permisos por rol

```
.authorizeHttpRequests(peticiones -> peticiones
        .requestMatchers("/", "/index.html", "/*.html",
                "/css/**", "/img/**", "/js/**").permitAll()
        .requestMatchers("/api/auth/login").permitAll()
        .requestMatchers(HttpMethod.DELETE, "/api/**").hasRole("ADMIN")
        .requestMatchers("/api/**").authenticated()
        .anyRequest().permitAll())
```

Las reglas se leen **de arriba hacia abajo** y se aplica la primera que coincide. Por eso el orden importa: si la regla general de `/api/**` estuviera antes que la de `DELETE`, los permisos de administrador nunca se verificarían.

Rol	Puede hacer
ADMIN	Todo, incluida la eliminación de registros
RECEPCION	Consultar, crear y modificar
PROFESIONAL	Consultar, crear y modificar

Otras protecciones

Amenaza	Cómo se evita
Inyección SQL	Todas las consultas usan parámetros de JPA; nunca se concatena texto
XSS (scripts inyectados)	La función `esc()` del frontend convierte los caracteres peligrosos antes de insertar datos en la página
Robo de contraseñas	Hash BCrypt con salt
Token adulterado	Firma con clave secreta del servidor
Sesión eterna	El token vence a las 8 horas

13. El frontend y cómo consume la API

El archivo central: js/api.js

Todas las páginas se comunican con el backend a través de este archivo. Ninguna llama a `fetch` por su cuenta.

```
async function peticion(metodo, ruta, cuerpo) {

    const opciones = {
        method: metodo,
        headers: { "Content-Type": "application/json" }
    };

    const token = obtenerToken();
    if (token) {
        opciones.headers["Authorization"] = "Bearer " + token;
    }

    if (cuerpo !== undefined) {
        opciones.body = JSON.stringify(cuerpo);
    }

    const respuesta = await fetch(API + ruta, opciones);

    if (respuesta.status === 401) {
        cerrarSesion();
        throw new ErrorApi("Su sesión expiró, vuelva a iniciar sesión", 401);
    }

    if (respuesta.status === 204) return null;

    const texto = await respuesta.text();
    const datos = texto ? JSON.parse(texto) : null;

    if (!respuesta.ok) {
        const mensaje = (datos && datos.mensaje) || "No se pudo completar la operación";
        throw new ErrorApi(mensaje, respuesta.status, datos && datos.errores);
    }

    return datos;
}
```

> **Por qué centralizar.** Si cada página hablara con el servidor por su cuenta, el código para agregar el token estaría repetido diez veces. Al estar en un solo lugar, un cambio se hace una sola vez. Además, el control del 401 cierra la sesión automáticamente desde cualquier página.

Qué es async / await

Pedir datos al servidor **lleva tiempo**. Si el navegador se quedara esperando, la página se congelaría. Con `await` el navegador sigue respondiendo mientras espera, y continúa el código cuando llega la respuesta.

Un ejemplo de uso: el listado de pacientes

```
async function cargarPacientes() {

    const texto = campoBusqueda.value.trim();
    const ruta = texto ? `/pacientes?buscar=${encodeURIComponent(texto)}` : "/pacientes";

    cuerpoTabla.innerHTML = `<tr><td colspan="6">Cargando...</td></tr>`;

    try {
        const pacientes = await api.get(ruta);
        dibujarPacientes(pacientes);
    } catch (error) {
        manejarError(error);
    }
}

function dibujarPacientes(pacientes) {
    cuerpoTabla.innerHTML = pacientes.map(p => `
        <tr>
            <td>${esc(p.dni)}</td>
            <td>${esc(p.apellido)}</td>
            <td>${esc(p.nombre)}</td>
            <td class="text-center">
                <a href="paciente-form.html?id=${p.id}" class="btn btn-sm btn-outline-
primary">Editar</a>
                <button data-eliminar="${p.id}" class="btn btn-sm btn-outline-
danger">Eliminar</button>
            </td>
        </tr>
    `).join("");
}
```

La función `encodeURIComponent` es necesaria porque el texto buscado puede tener espacios o acentos, que no son válidos tal cual dentro de una dirección web.

La protección contra XSS

```
function esc(valor) {
    if (valor === null || valor === undefined) return "";
    return String(valor)
        .replace(/&/g, "&amp;")
        .replace(/</g, "&lt;")
        .replace(/>/g, "&gt;")
        .replace(/"/g, "&quot;");
}
```

Si alguien guardara `<script>...</script>` como nombre de un paciente, sin esta función el navegador lo ejecutaría al dibujar la tabla. Con `esc()` se muestra como texto común.

El control de sesión en cada página

```
function exigirSesion() {
    if (!obtenerToken()) {
        window.location.href = "index.html";
        return false;
    }
    return true;
}
```

Cada página privada llama a esta función al cargar. Si no hay token, vuelve al login.

Las páginas y su archivo JavaScript

Página	Archivo	Qué hace
index.html	login.js	Envía las credenciales y guarda el token
dashboard.html	dashboard.js	Muestra los totales del sistema
pacientes.html	pacientes.js	Lista, busca y elimina pacientes
paciente-form.html	paciente-form.js	Alta y edición de pacientes
profesionales.html	profesionales.js	Lista y filtra por especialidad
profesional-form.html	profesional-form.js	Alta y edición de profesionales
especialidades.html	especialidades.js	ABM con ventana emergente
turnos.html	turnos.js	Filtros, alta y cambio de estado
agenda.html	agenda.js	Arma la grilla semanal
historia-clinica.html	historia-clinica.js	Fichas e historial de atenciones

Cómo se arma la agenda

La agenda pide los turnos de la semana y los organiza por fecha y hora para ubicarlos en la grilla:

```
const turnos = await api.get(
    `/turnos/agenda?
profesionalId=${profesionalId}&desde=${aISO(lunes)}&hasta=${aISO(viernes)}`);

// Se indexan los turnos por "fecha|hora" para ubicarlos en la grilla
const porCelda = {};
turnos.forEach(t => { porCelda[`${t.fecha}|${t.hora}`] = t; });
```

Después, al dibujar cada celda, se consulta si existe un turno en esa combinación. Si existe se muestra el paciente; si no, un botón de horario libre. Esta técnica evita recorrer toda la lista de turnos por cada celda.

El diseño responsive

El sistema se adapta a celulares mediante las clases de Bootstrap (`col-12 col-md-6 col-lg-4`), que apilan las columnas en pantallas chicas. Las tablas se desplazan horizontalmente dentro de `.table-responsive` en lugar de comprimirse, y la agenda cambia a una vista de tarjetas por día.

14. Tabla completa de endpoints

Todos requieren el encabezado `Authorization: Bearer <token>`, excepto el login.

Autenticación

Método	Ruta	Descripción
POST	/api/auth/login	Devuelve el token JWT

Pacientes

Método	Ruta	Descripción
GET	/api/pacientes?buscar=texto	Lista y busca por DNI, nombre o apellido
GET	/api/pacientes/{id}	Obtiene un paciente
GET	/api/pacientes/dni/{dni}	Busca por documento
POST	/api/pacientes	Registra un paciente
PUT	/api/pacientes/{id}	Modifica un paciente
DELETE	/api/pacientes/{id}	Elimina (solo ADMIN)

Especialidades

Método	Ruta	Descripción
GET	/api/especialidades?buscar=&soloActivas=	Lista y busca
GET	/api/especialidades/{id}	Obtiene una especialidad
POST	/api/especialidades	Crea una especialidad
PUT	/api/especialidades/{id}	Modifica una especialidad
DELETE	/api/especialidades/{id}	Elimina (solo ADMIN)

Profesionales

Método	Ruta	Descripción
GET	/api/profesionales?buscar=&especialidadId=&soloActivos=	Lista y filtra
GET	/api/profesionales/{id}	Obtiene un profesional
POST	/api/profesionales	Registra un profesional
PUT	/api/profesionales/{id}	Modifica un profesional
DELETE	/api/profesionales/{id}	Elimina (solo ADMIN)

Turnos

Método	Ruta	Descripción
GET	/api/turnos?buscar=&profesionalId=&fecha=&estado=	Lista con filtros combinables
GET	/api/turnos/agenda?profesionalId=&desde=&hasta=	Turnos para la agenda semanal
GET	/api/turnos/{id}	Obtiene un turno
POST	/api/turnos	Registra un turno
PUT	/api/turnos/{id}	Modifica un turno
PATCH	/api/turnos/{id}/estado	Cambia el estado
DELETE	/api/turnos/{id}	Elimina (solo ADMIN)

Historia clínica y panel

Método	Ruta	Descripción
GET	/api/atenciones?pacienteId={id}	Historial del paciente, de la más reciente a la más antigua
GET	/api/atenciones/{id}	Obtiene una atención
POST	/api/atenciones	Registra una atención
DELETE	/api/atenciones/{id}	Elimina (solo ADMIN)
GET	/api/dashboard	Totales y estadísticas

Ejemplo de llamada y respuesta

Pedido:

```
POST /api/auth/login
Content-Type: application/json

{ "username": "admin", "password": "admin123" }
```

Respuesta:

```
200 OK

{
  "token": "eyJhbGciOiJIUzUxMiJ9...",
  "username": "admin",
  "nombreCompleto": "Administrador del Sistema",
  "rol": "ADMIN"
}
```

15. La base de datos

Las tablas

Tabla	Contenido	Relaciones
usuarios	Credenciales y rol	—
pacientes	Datos personales, obra social, grupo sanguíneo, alergias	—
especialidades	Nombre, descripción y estado	—
profesionales	Matrícula, nombre, contacto, estado	Pertenece a una especialidad
turnos	Fecha, hora, estado, motivo	Pertenece a un paciente y a un profesional
atenciones	Motivo, síntomas, diagnóstico, tratamiento, indicaciones	Pertenece a un paciente y a un profesional

Diagrama de relaciones

```
especialidades
      |
      | 1 a muchos
      v
profesionales  -------+
      |               |
      | 1 a muchos    | 1 a muchos
      v               v
   turnos   <----  atenciones
      ^               ^
      |               |
      +------ pacientes ------+
       (1 a muchos en ambas)
```

La configuración de la conexión

```
spring.datasource.url=jdbc:postgresql://localhost:5432/centro_medico
spring.datasource.username=${DB_USER:postgres}
spring.datasource.password=${DB_PASSWORD:...}

spring.jpa.hibernate.ddl-auto=update
```

La notación `${DB_USER:postgres}` significa: usar la variable de entorno `DB_USER` y, si no existe, usar `postgres`. Así los datos sensibles pueden configurarse por fuera del código.

`ddl-auto=update` hace que Hibernate cree o actualice las tablas según las entidades al arrancar. Es cómodo para desarrollo; en un sistema en producción se usarían scripts de migración controlados.

Los datos iniciales

La clase `DataSeeder` carga datos de ejemplo la primera vez:

```
@Override
public void run(String... args) {
    if (usuarioRepository.count() > 0) {
        return;     // la base ya tiene datos, no hace nada
    }
    crearUsuarios();
    // especialidades, profesionales, pacientes, turnos, atenciones...
}
```

Implementa `CommandLineRunner`, una interfaz que Spring ejecuta automáticamente al terminar de arrancar.

Usuarios de prueba

Usuario	Contraseña	Rol
admin	admin123	ADMIN
recepcion	recepcion123	RECEPCION
leiva	leiva123	PROFESIONAL

16. Preguntas probables en la defensa

¿Por qué separaste el proyecto en capas?

Para que cada parte tenga una sola responsabilidad. El controlador recibe peticiones, el servicio decide y el repositorio consulta la base. Así, un cambio en la base de datos no obliga a tocar los controladores, y la lógica de negocio se puede probar sin base de datos.

¿Qué es un DTO y por qué no devolvés las entidades?

Un DTO define exactamente qué entra y qué sale de la API. No devuelvo las entidades porque, por ejemplo, `Usuario` contiene el hash de la contraseña y saldría en la respuesta. Además los DTO permiten agregar datos calculados como la edad, que no está guardada en ninguna columna.

¿Dónde validás los datos?

En dos lugares con propósitos distintos. El formato se valida en los DTO con anotaciones, y responde si el dato tiene la forma correcta. Las reglas de negocio se validan en los servicios, porque necesitan consultar la base: por ejemplo, si ese DNI ya existe. Lo primero devuelve 400 y lo segundo 409.

¿Cómo funciona la autenticación?

Con JWT. Al iniciar sesión se compara el hash BCrypt de la contraseña y, si coincide, se devuelve un token firmado que contiene el usuario y su rol. El navegador lo guarda y lo envía en cada petición; un filtro lo valida antes de dejar pasar la petición. El servidor no guarda sesiones, cada pedido se valida por su firma.

¿El token es seguro si cualquiera puede leerlo?

Sí, porque está firmado, no encriptado. Se puede leer su contenido pero no modificarlo: la firma se genera con una clave secreta que solo tiene el servidor, y cualquier alteración la invalida. Por eso dentro del token no se guarda información sensible.

¿Cómo evitás la inyección SQL?

Todas las consultas pasan por JPA con parámetros, nunca se concatena texto del usuario dentro de una consulta. Lo que se escribe se trata siempre como dato, nunca como instrucción SQL.

¿Cómo impedís dos turnos en el mismo horario?

En dos niveles. El servicio consulta antes de guardar si ese profesional ya tiene un turno en esa fecha y hora, ignorando los cancelados. Y además la tabla tiene una restricción de unicidad sobre profesional, fecha y hora, así que aunque alguien escribiera directo en la base, no podría duplicarlo.

¿Por qué al eliminar un paciente a veces no se borra?

Porque si tiene turnos o atenciones, borrarlo destruiría información médica. En ese caso se lo marca como inactivo: desaparece de los listados pero su historia clínica queda. Si no tiene nada asociado, se elimina de verdad.

¿Qué diferencia hay entre PUT y PATCH?

PUT reemplaza el registro completo, por eso al editar un turno se envían todos sus datos. PATCH modifica solo una parte: lo uso para cambiar únicamente el estado del turno, en `/api/turnos/{id}/estado`.

¿Qué pasa si el token vence mientras el usuario trabaja?

La API responde 401 y el archivo `api.js` detecta ese código, borra los datos guardados y redirige al login automáticamente, desde cualquier página.

¿Por qué el frontend está dentro del proyecto de Spring Boot?

Porque Spring Boot publica lo que está en `resources/static`. Así se levanta todo con un solo comando y no hay problemas de CORS, ya que frontend y API comparten la misma dirección. De todos modos dejé CORS configurado para localhost por si se quisiera servir el frontend por separado.

¿Qué es inyección de dependencias?

Es que Spring crea los objetos y se los entrega a las clases que los necesitan, en lugar de que cada clase los construya. Por eso los servicios se reciben por constructor: la clase declara qué necesita y Spring se lo provee. Eso también permite reemplazarlos por simulaciones al hacer pruebas.

¿Qué hace @Transactional?

Agrupa las operaciones del método en una transacción: o se aplican todas o ninguna. Si el método falla a la mitad, la base vuelve atrás y no quedan datos inconsistentes.

¿Es responsive?

Sí. Usa el sistema de columnas de Bootstrap, que apila el contenido en pantallas chicas. Las tablas se desplazan horizontalmente en lugar de comprimirse, y la agenda semanal cambia a una vista de tarjetas por día en el celular.